ML CHALLENGES IN REAL WORLD
(AI_541_001_S2025)

Diabetes Diagnosis in Iraq

<u>Final Report and Lessons Learned</u>

Shivani Gadipe
Date: 06/10/2025

Table of Content:

Content	Page.no
Problem to be solved	3
Data set properties	3
ML model(s)	4
Evaluation	5
Challenges and solutions	6
Experimental results	8
Reflection	11
Response to draft report	11

1. Problem to be solved:

Problem:

Diabetes is a long term health condition that affects how the body uses sugar. If not diagnosed and managed early, it can lead to serious health problems such as heart disease, kidney failure, and vision loss. The goal of this project is to build a machine learning model that can predict whether a person has diabetes based on simple health indicators.

Beneficiaries/Users:

- Doctors and healthcare professionals to support early screening and diagnosis.
- Patients to help them understand their health risks.
- Health organizations to plan community health programs and preventive measures.

This project contributes to public health by enabling faster and more accessible diabetes prediction using machine learning.

2. Data set properties:

Source: Mendeley Data

URL: https://data.mendeley.com/datasets/wj9rwkp9c2/1

Author: Ahlam Rashid (2020)

Dataset Profile:

Total Records: 1000

Number of Features: 14

Outcome (N = Non-Diabetic, P = Diabetic, Y = Pre Diabetic)

Non-Diabetic = 103

Diabetic = 53

Pre Diabetic = 844

Missing Values: There are no missing values in the dataset.

Numeric Features:

Feature	Min	Max	Mean	Median
ID	1	800	340.5	300.5
No_pation	123	75435657	270551.41	34395.5
AGE	20	79	53.528	55
Urea	0.5	38.9	5.124743	4.6
Cr	6	800	68.943	60
HbA1c	0.9	16	8.28116	8
Chol	0	10.3	4.86282	4.8
TG	0.3	13.8	2.34961	2
HDL	0.2	9.9	1.20475	1.1
LDL	0.3	9.9	2.60979	2.5
VLDL	0.1	35	1.8547	0.9
BMI	19	47.75	29.57802	30

This table shows the min, max, mean, median values for each feature in the data. Important features like HbA1c, BMI, and Age vary a lot from person to person, which helps the model figure out who might have diabetes. For example, many people in the dataset have high HbA1c and BMI levels and that's something we usually see in diabetic patients. Some columns like ID and No_pation are just for identification and won't be used in training the model.

3. Machine learning models:

In this project, I used three classifiers:

- **Random Forest**: A powerful ensemble model that handles non-linear relationships and feature interactions well.
- **Logistic Regression**: A simple, interpretable model, especially helpful when using class weighting.
- **Support Vector Machine (SVM)**: Used mainly for comparison in under-sampling and other strategy tests.

Strengths and Weaknesses

- **Random Forest** is robust and performs well but can overfit if the data is noisy.
- **Logistic Regression** is easy to understand and fast to train but may struggle with non-linear patterns.
- **SVM** is effective on smaller datasets but is sensitive to feature scaling and can be slow on large datasets.

Hyperparameters and Settings

- **Random Forest**:
  n_estimators = 100 (number of trees)
  criterion = 'gini' (splitting strategy)
  max_depth = None (allowing trees to grow fully)
  min_samples_leaf = 1 (default)
  class_weight = 'balanced' (to handle class imbalance)
- **Logistic Regression**:
  max_iter = 1000 (to ensure convergence)
  class_weight = 'balanced'
- **SVM**:
  C = 1.0 (regularization strength)
  kernel = 'rbf' (default kernel)
  class_weight = 'balanced'

4. Evaluation:

To evaluate how well each model performed, I used four key metrics:

- **Accuracy:** Measures how many predictions were correct overall.
- Precision: Tells how many predicted labels were actually correct, taking false positives into account.
- **Recall:** Tells how many actual class labels were correctly found by the model, focusing on false negatives.
- **F1 Score:** A balance between precision and recall especially useful for imbalanced datasets.

Since my dataset has three classes (Non-Diabetic, Pre-Diabetic, Diabetic), I used weighted averaging for precision, recall, and F1 Score. This way, the evaluation reflects the performance across all classes proportionally.

Train/Test/Validation Setup:

I split my dataset into three parts:

- Training set (70%) - used to train the model
- Validation set (15%) - used internally to fine-tune and test strategies
- Test set (15%) - used to report final model performance

All strategies like SMOTE, under-sampling, and scaling were applied only on the training data, and the test data remained untouched to get a fair evaluation.

Baseline:

I included a baseline model using DummyClassifier that always predicts the majority class (Pre-Diabetic). This gave much lower scores compared to my trained models, showing that the real models learned meaningful patterns rather than just memorizing the common label.

5. Challenges and Solutions:

Challenge 1: Class Imbalance

In my dataset, out of 1000 patients, 844 were Pre-Diabetic, while only 103 were Non-Diabetic and 53 were Diabetic. This imbalance made the model biased toward predicting the Pre-Diabetic class more often.

Strategy 1: SMOTE

I used SMOTE (Synthetic Minority Oversampling Technique) to generate new synthetic examples for the minority classes (Non-Diabetic and Diabetic). I applied SMOTE only on the training set, and left the test set unchanged. This helped balance the data during training and improve model learning.

Citation: Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. Journal of Artificial Intelligence Research, 16, 321-357. https://doi.org/10.1613/jair.953

Strategy 2: Under-Sampling

Here, I reduced the number of Pre-Diabetic samples in the training set to match the number of samples from the minority classes. This forced the model to learn from a more balanced dataset. Again, I only applied this to the training set.

Strategy 3: Class Weighting

I used class weights to give more importance to the minority classes. I computed these weights from the training data using compute_class_weight. These were passed as hyperparameters to the classifiers that support it (Logistic Regression, SVM, Random Forest). This helped the model not ignore small classes.

Challenge 2: Outliers

Some features, like Creatinine (Cr) and Triglycerides (TG), had extremely large values. For example, Cr ranged from 6 to 800, which could confuse the model and negatively affect learning.

Strategy 1: Winsorization

I capped the values at the 1st and 99th percentiles, so that extreme values would not dominate training. This transformation was applied on all features in the training data and reused for test data scaling.

Strategy 2: Z-score Filtering

I calculated Z-scores for each record and removed rows with values more than 3 standard deviations away from the mean in any feature. This helped remove extreme outliers from the training data only.

Strategy 3: Robust Scaling

I used RobustScaler, which scales the data using the median and interquartile range (IQR). This is less affected by outliers. I fitted the scalar only on the training data and used the same scaler to transform the test set.

Challenge 3: Features with Different Scales

My features had very different scales. For example, AGE ranged from 20 to 79, while HDL ranged from 0.2 to 9.9. If not scaled, larger values could dominate smaller ones and confuse the model.

Strategy 1: Min-Max Scaling

I scaled all features between 0 and 1 using MinMaxScaler. This helped the model treat all features equally.

Strategy 2: Standardization

I applied StandardScaler to make all features have a mean of 0 and standard deviation of 1. This helped models like SVM and Logistic Regression.

Strategy 3: Robust Scaling

I reused RobustScaler here too, as it not only handles different feature scales but also deals well with outliers.

6. Results:

Evaluated the performance of three classifiers Random Forest, Logistic Regression, and SVM on the diabetes dataset across three key challenges: class imbalance, outliers, and feature scaling. For each challenge, we applied three strategies and compared them to a baseline model and raw model performance (i.e., without any preprocessing).

All experiments used a consistent training/test split and evaluation was based on Accuracy, Precision, Recall, and F1 Score, using weighted averages due to class imbalance.

Raw performance before any strategy:

Model	Accuracy	Precision	Recall	F1 Score
Random Forest	0.9933	0.9934	0.9933	0.9931
Logistic Regression	0.9333	0.9293	0.9333	0.9307
SVM	0.8400	0.7056	0.8400	0.7670

This shows the performance of each model on the unmodified dataset with no preprocessing. Random Forest performed the best even without preprocessing.

Challenge 1: Class Imbalance

Strategy	Model	Accuracy	Precision	Recall	F1 Score
SMOTE	RandomForest	0.9933	0.9934	0.9933	0.9931
SMOTE	LogisticRegression	0.9200	0.9368	0.9200	0.9273
SMOTE	SVM	0.9533	0.9599	0.9533	0.9555
UnderSampling	RandomForest	0.9400	0.9594	0.9400	0.9454
UnderSampling	LogisticRegression	0.8933	0.9560	0.8933	0.9153
UnderSampling	SVM	0.9067	0.9582	0.9067	0.9247
ClassWeighting	RandomForest	0.9933	0.9934	0.9933	0.9931
ClassWeighting	LogisticRegression	0.9267	0.9182	0.9267	0.9221
ClassWeighting	SVM	0.9400	0.9328	0.9400	0.9347

Best Strategy: SMOTE with Random Forest gave the best results overall with an F1 score of 0.9931, matching the raw model. Class weighting also performed very well with both Random Forest and SVM. Under-sampling reduced overall performance slightly but was still better than the baseline. The baseline classifier showed a much lower F1 score, proving the effectiveness of these strategies.

Challenge 2: Outliers

Strategy	Model	Accuracy	Precision	Recall	F1 Score
Winsorization	RandomForest	0.9733	0.9738	0.9733	0.9724
Winsorization	LogisticRegression	0.9200	0.9009	0.9200	0.9089
Winsorization	SVM	0.9400	0.9328	0.9400	0.9347
Z-score	RandomForest	0.9733	0.9736	0.9733	0.9730
Z-score	LogisticRegression	0.9400	0.9304	0.9400	0.9339
Z-score	SVM	0.9400	0.9282	0.9400	0.9317
Robust Scaling	RandomForest	0.9933	0.9934	0.9933	0.9931

Robust Scaling	LogisticRegression	0.9267	0.9086	0.9267	0.9163
Robust Scaling	SVM	0.9267	0.8784	0.9267	0.9018

Best Strategy: Among outlier handling methods, Robust Scaling with Random Forest again matched the best F1 score of 0.9931. The other methods Winsorization and Z-score provided modest improvements for Logistic Regression and SVM. These strategies were applied on the original training data before SMOTE and scaled using parameters fit only on the training set.

Challenge 3: Feature Scaling

Strategy	Model	Accuracy	Precision	Recall	F1 Score
MinMax	RandomForest	0.9933	0.9934	0.9933	0.9931
MinMax	LogisticRegression	0.9200	0.8702	0.9200	0.8944
MinMax	SVM	0.9333	0.8857	0.9333	0.9085
Standard	RandomForest	0.9933	0.9934	0.9933	0.9931
Standard	LogisticRegression	0.9267	0.9182	0.9267	0.9221
Standard	SVM	0.9400	0.9328	0.9400	0.9347
Robust	RandomForest	0.9933	0.9934	0.9933	0.9931
Robust	LogisticRegression	0.9267	0.9086	0.9267	0.9163
Robust	SVM	0.9267	0.8784	0.9267	0.9018

Best Strategy: All three scaling methods helped improve SVM and Logistic Regression performance, which are sensitive to feature scales. Random Forest remained robust and performed the same across all strategies, indicating it handles unscaled features well. Standard and Robust scaling gave the most consistent improvement for the other models.

7. Reflections:

Working on this project taught me that applying machine learning to real-world data isn't just about writing code, it's about solving messy problems like imbalanced classes, scaling issues, and outliers. I was surprised that my Random Forest model gave such high accuracy (around 99%), and I wondered at first if it was overfitting. But after checking other metrics like the F1 score and comparing with a baseline, I realized it was genuinely doing well because the data was clean and the classes were fairly distinguishable.

I showed my project results to classmate **Sanika**, and she pointed out that the "Raw-No Strategy" of Logistic Regression actually performed better than when using class weighting. She wondered if this might be because Logistic Regression uses a liblinear solver, which might not always handle class weighting well. This made me think more about how the solver and algorithm settings can impact results, even when we expect a strategy like class weighting to help.

From my original plan, I changed a lot. I initially thought I would only use Random Forest, but later decided to include Logistic Regression and SVM for comparison.

If I had more time, I would try using deep learning models or ensemble techniques to see if they could push the performance even further. Additionally, I'd like to test the model on new patient data to see how well it performs in real-world settings.

R. Response to draft report feedback:

1. Comment on whether there are any missing values.

Ans: Already mentioned in the draft report, now I have highlighted it in the final report.
There are **no missing values** in the dataset.

2. Include more of the important hyperparameters for each model (RF: n_estimators, criterion, max_depth, min_samples_leaf; SVM: C, kernel (and associated parameters))

Ans: In my final report, I added a dedicated section describing the key hyperparameters used for each model. For Random Forest, I included n_estimators=100, criterion='gini', max_depth=None, and min_samples_leaf=1, explaining that these are common defaults and allow the model to fully capture patterns in the data. For SVM, I specified C=1.0 and kernel='rbf' as they are standard and suitable for non-linear classification tasks. For Logistic Regression, I highlighted the use of class_weight='balanced' to address class imbalance.

3. Explain how you chose these values.

Ans: I explained that I used commonly accepted default values provided by scikit-learn because they generally perform well and offer a good starting point. My project focus was on comparing preprocessing strategies across models rather than tuning the models themselves. I also mentioned that due to time constraints and the scope of the project, I did not perform an extensive hyperparameter search but prioritized consistency across models for fair comparison.

5. Since you have 3 classes, explain how precision, recall, and F1 are measured.

Ans: Since my project had three classes Non-Diabetic, Pre-Diabetic, and Diabetic. I used weighted averages for precision, recall, and F1 score. That means it calculates the scores for each class and then averages them, giving more weight to classes that have more samples. I used average='weighted' in scikit-learn so the final scores reflect how well the model performs across all classes, not just the most common one.

4. Explain that your baseline uses pre-diabetic because it is the majority in the training set.

Ans: For the baseline model, I used a simple strategy that always predicts the Pre-Diabetic class. I chose this because Pre-Diabetic was the most common class in the training data. This helps set a basic reference point. Any good model should perform better than just always picking the most frequent class.

5. Ensure that SMOTE and under-sampling are applied only to the training data. The test set should stay the same for all strategies.

Ans: Yes, I made sure that both SMOTE and under-sampling were applied only to the training data. I did not touch the test set, so it stayed the same for all strategies. This way, I could fairly compare how each method performed on unseen data without introducing bias.

For outlier handling and feature scaling, I followed the correct approach too. I fitted the transformation like scaling or winsorizing only on the training data, and then applied the same transformation to the test set. This makes sure the test set stays unseen and realistic.

6. More details needed - how many SMOTE examples will you create? How much will you under-sample given you have 3 classes? What weights will you use? I gave you this feedback on your initial findings document.

Ans: After applying SMOTE, the minority classes (Non-Diabetic and Diabetic) were upsampled to match the number of Pre-Diabetic samples (844), so all classes had equal representation.
For under-sampling, I reduced the Pre-Diabetic samples down to match the smallest class, which was Diabetic (53 samples). So in this case, all three classes had 53 samples after under-sampling.

For class weighting, I used compute_class_weight from scikit-learn, which automatically assigns higher weights to the minority classes based on how imbalanced the original training set was.

7. Add a citation for SMOTE.

Ans: Now I have included SOMTE citation in this final documentation:

Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. Journal of Artificial Intelligence Research, 16, 321-357. https://doi.org/10.1613/jair.953

8. Clarify which features you applied the outlier strategies to (only Cr and TG, or all features?).

Ans: I applied outlier detection and correction strategies like winsorization and z-score filtering to all numerical features, not just Cr and TG. This helped in removing extreme values that could affect model training.

9. Clarify which features you applied feature scaling to (only AGE and HDL, or all features?).

Ans: I applied feature scaling (MinMax, Standard, Robust) to all numerical features, not just AGE and HDL. Since different features had very different ranges, I wanted to make sure scaling was consistent across all inputs to improve model performance.

10. Indicate which model was used for these results. Your plan says you will use RF, LR, and SVM.

Ans: Now I have mentioned in the final report. I have used all the 3 models for all the 3 strategies and I mentioned which strategy with which model gave the best performance.

11. Show results for each of the model types.

Ans: Yes, in my final report, I included results for all three models (RF, LR, and SVM) under each challenge and every strategy like SMOTE, under-sampling, class weighting, etc. This helped me understand which combination worked best.

12. Include the majority-class baseline result in each table.

Ans: I included a DummyClassifier as a baseline that always predicts the majority class (Pre-Diabetic). This gave much lower performance, especially in precision and F1 score. It

helped highlight how much better my actual models performed compared to guessing the most common class.

13. In each table, show original performance before any strategies are applied.

Ans: Yes, I added a "Raw Model Results" table that shows how each model performed before applying any strategies. This gives a clear reference to compare how much each strategy improved the model.

14. Outliers: winsorization and robust scaling did not seem to add any benefit over the SMOTE strategy. Clarify whether these results operate on the original training data, or the training data after SMOTE is done, or something else.

Ans: All challenges were treated independently. That means:

For outlier handling and scaling, I used the original training data, not the data after SMOTE or any other resampling.

Each strategy was applied on its own fresh copy of training data with the test set unchanged across all runs.

15. It sounds like you have each challenge build on the best strategy from an earlier challenge. If so, explain this explicitly. Following suggestion #3 will help make this clear too.

Ans: Actually, I did not build one challenge on top of another.

Each challenge class imbalance, outliers, and scaling was treated as a separate experiment, starting from the same original training data.

This helped me compare each strategy's impact on its own and avoid overlapping effects from other transformations.

16. Discuss strategies vs. original performance (it is possible for strategies to make things worse! If so, comment on this).

Ans: In my project, I compared each strategy with the original model performance (without applying any preprocessing).

- For Random Forest, the original performance was already very high (F1 = 0.9931), and applying strategies like SMOTE or Class Weighting didn't improve it much, they just maintained that performance.

- For Logistic Regression and SVM, some strategies improved performance like SMOTE and Class Weighting, especially in handling the class imbalance.

- However, not all strategies helped for example, Winsorization and Z-score outlier filtering sometimes reduced performance slightly compared to SMOTE or raw results. This shows that preprocessing doesn't always lead to improvements and must be chosen carefully.

17. Including results for multiple classifiers will give you more to talk about. The analysis of your results should be extended.

Ans: Yes, I tested three classifiers Random Forest, Logistic Regression, and SVM with each strategy.
This helped me see how different models react to the same challenge.
for example:

- SMOTE improved SVM's F1 score significantly from 0.767 (baseline) to 0.9555.

- Class Weighting worked well for all models, especially Logistic Regression.

- Random Forest stayed consistent and performed well even without any preprocessing. Having results for multiple classifiers helped me understand that some models are more sensitive to preprocessing than others.

18. Look at a confusion matrix and discuss what kinds of errors are most common.

Ans: From the confusion matrix I noticed that:

Pre-Diabetic cases, which is a majority class, were almost always predicted correctly.

The minority classes, especially Diabetic (P), were more likely to be misclassified, often confused with Pre-Diabetic.
This confirms that the model was initially biased toward the majority class.
Strategies like SMOTE helped by generating more synthetic examples of the underrepresented classes, which reduced this error and improved balance across predictions.